

Consumindo APIs no Front-end

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br

Sumário

- API (Application Programming Interface)
- Protocolo HTTP
- EndPoint
- JSON (JavaScript Object Notation)
- Atividade 01
- Servidor Backend e Web Service
- Criando uma API REST com Express
- Atividade 02

API (Application Programming Interface)

Uma Interface de Programação de Aplicações é um conjunto de protocolos, rotinas e ferramentas para construção de software.

Ela define como diferentes componentes de software devem interagir, permitindo que sistemas distintos se comuniquem entre si.

API (Application Programming Interface)

REST (Representational State Transfer) - Transferência de Estado Representacional é um estilo arquitetural para desenvolvimento de sistemas distribuídos, especialmente aplicados na web. Seus princípios incluem:

- Comunicação cliente-servidor sem estado (stateless)
- Uso padrão de métodos HTTP
- Recursos identificados por URIs
- Representações de dados (como JSON)
- Métodos HTTP Principais

Protocolo HTTP

HTTP (Hypertext Transfer Protocol) é o protocolo que permite a comunicação na World Wide Web. Ele estabelece as regras para como clientes (como navegadores) e servidores trocam informações.

Protocolo HTTP

Conceitos principais:

- Modelo cliente-servidor: Seu navegador (cliente) faz requisições a servidores web
- Stateless: Cada requisição é independente (o servidor não "lembra" de requisições anteriores)
- Baseado em texto: As mensagens são legíveis por humanos

Métodos HTTP

- **GET**

- Finalidade: Recuperar informações do servidor
- Características: Seguro (não altera dados), idempotente (várias chamadas = mesmo resultado)

- **POST**

- Finalidade: Criar novos recursos no servidor
- Características: Não idempotente (chamadas repetidas criam múltiplos recursos)

- **PUT/PATCH**

- PUT: Substitui completamente um recurso existente
- PATCH: Atualiza parcialmente um recurso
- Características: Ambos servem para atualizar as informações

- **DELETE**

- Finalidade: Remover um recurso específico
- Características: Idempotente (apagar algo já apagado não causa erro)

Como funciona uma requisição na prática

1. Navegador / Front-end: Usuário acessa a página ou clica em um botão no sistema.
2. Requisição HTTP: O navegador envia uma requisição ao servidor (GET, POST, PUT, DELETE).
3. Servidor Express.js: O backend recebe a requisição, identifica a rota e executa a lógica necessária.

Como funciona uma requisição na prática

4. Banco de Dados / API Externa: O servidor busca, grava ou atualiza informações em banco de dados ou outros serviços.
5. Resposta JSON: Após processar a requisição, o servidor retorna os dados em formato JSON.
6. Atualização da Tela: O front-end recebe a resposta e exibe os dados ou mensagens ao usuário.

EndPoint

- Um endpoint é uma URL específica que fornece acesso a um recurso ou funcionalidade em uma API.
- Representa o ponto de comunicação entre o cliente e o servidor.
- **Exemplo:** <https://github.com/awesomeapibrasil/awesomeapi-cep>
 - Método GET: Lista todos os usuários.
 - Método POST: Adiciona um novo usuário.

Repositórios de APIs Públicas

Uma alternativa interessante para estudar e desenvolver aplicações reais é utilizar catálogos de APIs públicas. Eles reúnem APIs disponibilizadas por diferentes projetos e serviços, permitindo que o desenvolvedor encontre fontes de dados para integrar às suas aplicações.

Exemplo: <https://www.freepublicapis.com/>

JSON (JavaScript Object Notation)

- Formato leve de troca de dados que:
 - É fácil para humanos ler e escrever
 - É fácil para máquinas parsear e gerar
- Baseado em dois tipos de estruturas:
 - Coleções de pares nome/valor (objetos)
 - Listas ordenadas de valores (arrays)

```
1  {  
2    "id": "USER-001",  
3    "nome": "Carlos Silva",  
4    "email": "carlos@exemplo.com",  
5    "idade": 28,  
6    "ativo": true,  
7    "endereco": {  
8      "cidade": "São Paulo",  
9      "estado": "SP"  
10   },  
11   "interesses": ["tecnologia", "esportes"]  
12 }
```

```
1  {  
2    "frutas": ["maçã", "banana", "laranja"],  
3    "numeros": [1, 2, 3, 5, 8, 13],  
4    "booleanos": [true, false, true],  
5    "objetos": [  
6      {"id": 1, "nome": "item A"},  
7      {"id": 2, "nome": "item B"}  
8    ],  
9    "vazio": []  
10 }
```

Fonte: Elaborado pelo autor.

Atividade 01

Pesquise 10 projetos no GitHub que utilizem algum tipo de API em suas aplicações. Em seguida, clonem e analisem os projetos, identificando o framework utilizado e as APIs consumidas. Por fim, clonem os projetos e criem um arquivo Markdown com uma tabela detalhando os projetos escolhidos e suas respectivas informações.

CRIANDO UMA API

Servidor Backend e Web Service

- Servidor Backend: Sistema que processa requisições, gerencia dados e fornece respostas para clientes (apps, navegadores).
- Funções Principais:
 - Armazenar/recuperar dados (banco de dados)
 - Executar regras de negócio
 - Fornecer APIs para comunicação

Servidor Backend e Web Service

- Web Service: Serviço acessível via web que permite comunicação entre sistemas usando HTTP/HTTPS.
- Permite que sistemas heterogêneos (diferentes linguagens, plataformas ou tecnologias) se comuniquem de forma padronizada.

Criando uma API REST com Express

- # Passo 1: Inicializando o projeto
 - Crie uma nova pasta para seu projeto e abra no VS Code.
- # Passo 2: Instale o [Express.js](#)
 - `npm install express`
- # Passo 3: Instale o Cors
 - `npm install cors express`

Criando uma API REST com Express

- # Passo 4: Criando o arquivo [api.js](#)
 - Seguir o exemplo da imagem.
- # Passo 5: Executando o servidor
 - node api.js

CORS é um mecanismo de segurança que controla acesso entre domínios diferentes no navegador.

```
JS api.js > ...
1 import express from 'express';
2 import cors from 'cors';
3
4 const app = express();
5
6 app.use(cors());
7
8 app.get('/', (req, res) => {
9   res.json({
10     date: new Date().toLocaleString('pt-BR'),
11     status: 'API no Render funcionando!'
12   });
13 });
14
15 // Porta dinâmica para o Render
16 const PORT = process.env.PORT || 3000;
17 app.listen(PORT, () => {
18   console.log(`Servidor rodando na porta ${PORT}`);
19 });
```

Fonte: Elaborado pelo autor.

Framework Express.js

- O Express.js é um framework para Node.js que facilita a criação de servidores web e APIs (interfaces de programação de aplicações).
- Ele é minimalista, flexível e muito popular no ecossistema JavaScript.

Framework Express.js

- Simplifica o Node.js: O Node.js puro (http module) exige mais código para rotas, middlewares, etc. O Express torna isso mais fácil.
- Roteamento: Facilita a definição de rotas (ex: /users, /products).
- Middlewares: Funções que processam requisições/respostas (ex: autenticação, logs).
- Velocidade: Leve e rápido para criar APIs ou servidores web.

Node.js Puro vs Express.js

Figura 1: Código exemplo em Node.js

```
1  const http = require('http');
2
3  const server = http.createServer((req, res) => {
4    if (req.url === '/') {
5      res.end('Olá, mundo!');
6    } else {
7      res.end('Rota não encontrada!');
8    }
9  });
10
11 server.listen(3000);
```

Fonte: Elaboração própria.

Figura 2: Código exemplo em Express.js

```
1  const express = require('express');
2  const app = express();
3
4  app.get('/', (req, res) => {
5    res.send('Olá, mundo!');
6  });
7
8  app.listen(3000);
```

Fonte: Elaboração própria.

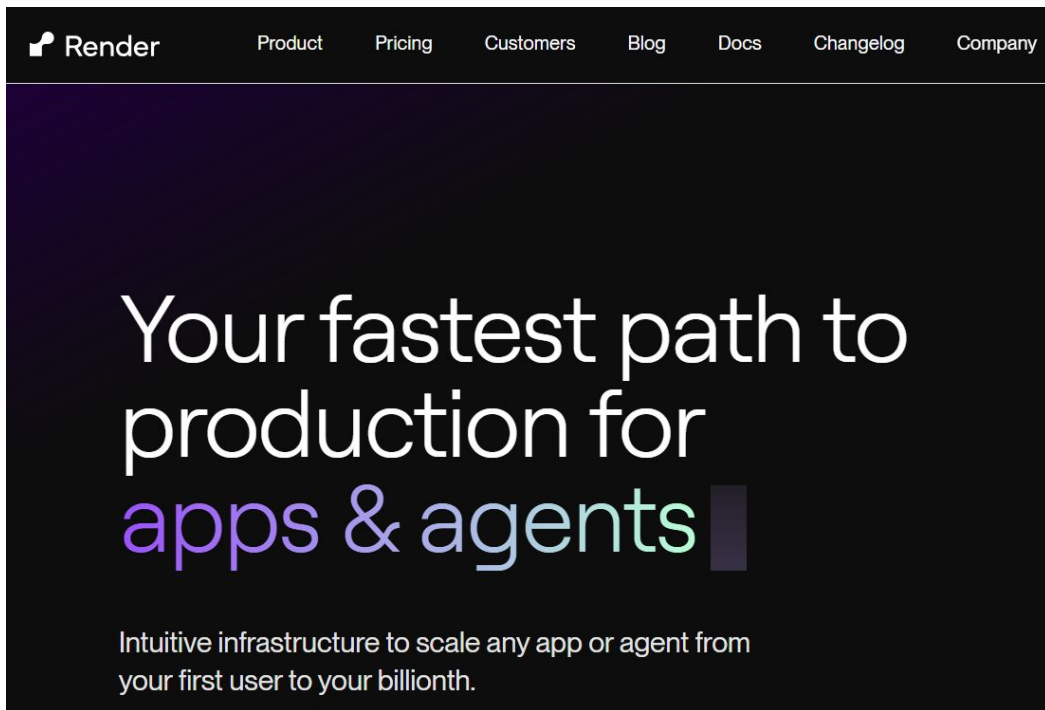
Quando usar Express.js?

- 1. Use Express.js para APIs REST eficientes, integração com bancos de dados e criação de backends escaláveis para aplicações web e mobile.
- 2. Ideal para servir páginas com templates e middleware que simplificam autenticação, logs e tratamento de erros.
- 3. Evite para aplicações em tempo real (prefira WebSockets) ou processamento pesado (use Workers), sendo melhor para prototipagem rápida.

Render para Simular Web Services

- Plataforma de hospedagem em nuvem moderna. Suporta Node.js, Python e outras linguagens. Integração fácil com repositórios Git.
- Oferece deploy contínuo automático. Planos gratuitos para projetos pequenos. Interface simples e intuitiva para iniciantes.
- Escalável para aplicações profissionais. Certificado SSL incluso gratuitamente. Ideal para APIs e microsserviços.

Render para Simular Web Services



Fonte: render.com

Benefícios do Render

- Deploy rápido em poucos cliques. Configuração simplificada sem terminal. Atualizações automáticas via GitHub.
- Ambiente de produção profissional. Escalabilidade automática sob demanda. Monitoramento de desempenho integrado.
- Suporte técnico eficiente. Infraestrutura confiável e segura. Perfeito para projetos acadêmicos.

Criando uma API REST com Express

- # Passo 1: Commit do Projeto no Github
 - Deixe disponível o projeto em um repositório do Github
- # Passo 2: Criando a conta no Render
 - Acesse dashboard.render.com
- # Passo 3: Crie novo "Web Service"
 - Clique no botão New e conecte o repositório no Github

Criando uma API REST com Express

- # Passo 4: Defina comando de start
 - Em Build Command: node / Em Start Command: node api.js
- # Passo 5: Deploy web service
 - Após realizar o deploy, utilize: seu-projeto.onrender.com

Atividade 02

1. Criem uma API usando Express, definindo uma rota de consulta de data e hora. Faça o deploy no Render, conectando ao repositório para garantir que a API fique acessível online. Em seguida, desenvolvam uma aplicação frontend que consuma essa API, e apresente na tela a data e hora.

Atividade 02

2. Utilize outro repositório para separar a API com o Front. Organize tudo em um documento com prints do código, da aplicação em funcionamento e dos painéis do Render e Vercel, links dos repositórios no GitHub.

Referências

1. SOUZA, Natan. ****Bootstrap 4: conheça a biblioteca front-end mais utilizada no mundo.**** São Paulo: Casa do Código, 2018. E-book. Disponível em: <https://plataforma.bvirtual.com.br>.
2. MACHADO, Kheronn Khennedy. ****Angular 11 e Firebase: construindo uma aplicação integrada com a plataforma do Google.**** São Paulo: Casa do Código, 2021. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 13 maio 2025.
3. EIS, Diego. ****Guia Front-end: o caminho das pedras para ser um dev front-end.**** São Paulo: Casa do Código, 2015. E-book. Disponível em: <https://plataforma.bvirtual.com.br>.
4. GONÇALVES, Edson. ****Desenvolvendo aplicações Web com JSP, Servlets, JavaServer Faces, Hibernate, EJB 3 Persistence e Ajax.**** Rio de Janeiro: Ciência Moderna, c2007.

Referências

5. HARTCOPP, Patrícia Ferreira. ****Métrica Web.**** São Paulo: Contentus, 2020. E-book (94 p.). Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/185191>. Acesso em: 30 abr. 2024.
6. NIEDERAUER, Juliano. ****Desenvolvendo Websites com PHP: aprenda a criar Websites dinâmicos e interativos com PHP e banco de dados.**** 3. ed. São Paulo: Novatec, 2017.
7. PREECE, J.; ROGERS, Y.; SHARP, H. ****Design de Interação: além da interação Homem-Computador.**** 3. ed. Porto Alegre: Bookman, 2013.
8. SOUSA, Roque Fernando Marcos. ****Canvas HTML 5: composição gráfica e interatividade na Web.**** Rio de Janeiro: Brasport, 2014. E-book (194 p.). Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/160686>. Acesso em: 22 jun. 2024.

Consumindo APIs no Front-end

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br